Phase 4: Candlestick Chart (REST history + live WebSocket)

What we built

- A **candlestick chart** (TradingView **Lightweight Charts**) in the middle panel, for the selected coin
- **History** loaded over **REST** (the last 500 candles), then **live** candles over the **WebSocket** (`btcusdt@kline_1m`)
- An **interval switcher**: 1m · 5m · 15m · 1h · 4h · 1d
- Loading and error overlays, **request cancellation** when you switch quickly, and local-time axis labels

Files and what each one does

File	Role
`src/services/binanceRest.ts`	`fetchCandles()` : an **async** function that calls `GET /api/v3/klines` , with host fallback and cancellation
`src/services/binanceMessages.ts`	+ kline parsing: the REST array rows and the WebSocket `kline` messages both become our `Candle` type
`src/components/CandleChart.tsx`	**The core.** Creates the chart in a `ref` 'd div, loads history, streams live candles
`src/components/ChartPanel.tsx`	Interval buttons (local `useState`) + the chart
`src/config.ts`	REST hosts, the interval list, the history size

Data flow

```
user picks BTCUSDT + 1m
        │
        ├── REST:  GET data-api.binance.vision/api/v3/klines?
symbol=BTCUSDT&interval=1m&limit=500
        │          → 500 candles → series.setData(...)                (the PAST,
once)
        │
        └── WS:    SUBSCRIBE btcusdt@kline_1m
                   → every ~2s: the current candle → series.update(...)  (the LIVE
edge)
                     same time as the last bar → replace it · newer time → append a
new bar
```

Concepts (theory only)

1. OHLC candles & intervals

One **candle** = what the price did during one **interval** (1 minute, 1 hour, 1 day…): - **O**pen: first trade price in the interval · **H**igh · **L**ow · **C**lose: last trade price - **Body**: from open to close. **Green** if close > open (price rose), **red** if it fell - **Wicks** (thin lines): stretch to the high and the low - The **latest candle is still forming**: its close (and maybe its high/low) changes on every trade until the interval ends. That's why live updates **replace** the last bar until a new interval starts.

2. REST vs WebSocket: history vs live

Need	Tool	Why
The last 500 candles	**REST** `GET /klines`	A one-time question with one answer
Every change to the current candle	**WebSocket** `@kline_1m`	The server pushes, with no polling

Real trading UIs always combine the two: **snapshot over REST, then deltas over WS**. (Phase 6's order book uses the same pattern, with sequence numbers.)

3. `fetch`, Promises, async/await

- **Promise** = an object for "a value that will arrive later". It's *pending*, then *fulfilled* (a value) or *rejected* (an error).
- `fetch(url)` returns a Promise of a **Response**. It resolves when the **headers** arrive. `res.json()` is a *second* Promise that reads and parses the **body**.
- `async function` always returns a Promise. Inside it, `await p` **pauses that function** (not the page!) until `p` settles. Errors become normal `try/catch`.
- ⚠ `fetch` **does not throw on 404/500**. It only throws on network failure. Check `res.ok` yourself.
- **Java analogy:** a Promise ≈ `CompletableFuture`. `await` ≈ `.get()` but **non-blocking**: the browser's single thread keeps handling clicks and rendering while it waits.

4. The race condition, and AbortController

Click BTC → ETH quickly: the BTC request might finish **after** the ETH one and draw BTC candles on the ETH chart. **Fix:** each effect run creates an `AbortController` and passes `controller.signal` to fetch. The effect's **cleanup** calls `controller.abort()`, so the old request is cancelled and its result is ignored. **A second race:** a live WS candle can arrive **before** the REST history. `setData()` would wipe it out. So we **buffer** early live candles and replay them after the history is drawn.

5. `useRef` for DOM access (and why charts need it)

React normally owns the DOM. But chart libraries draw on a `<canvas>` **themselves** and need a real element to mount into. `<div ref={containerRef} />` → after render, `containerRef.current` **is** that

`<div>`. We create the chart inside it in `useEffect` (the element only exists *after* rendering). We also keep the `chart` and `series` objects in refs: they must survive re-renders, and changing them must not cause one. This pattern (React gives a container, a third-party library takes over inside it) is called **integrating an imperative library**.

6. Two effects, two lifetimes

Effect	Dependencies	Setup	Cleanup
Create chart	`[]` → once	`createChart`, `addSeries`	`chart.remove()`
Load data	`[symbol, interval]`	fetch history + open kline socket	abort fetch + close socket

Splitting them means switching interval **reuses** the chart and only reloads data.

7. Live updates that skip React

`series.update(candle)` redraws the **canvas directly**: no `setState`, no re-render, no virtual-DOM diff. React only handles the “Loading…” overlay. For a high-frequency visual like a chart, that’s the fastest possible path, and it’s why it doesn’t need Phase 3’s rAF batching (the library already redraws efficiently).

8. Where state lives

The selected **interval** is local `useState` in `ChartPanel`: nothing else needs it. The selected **symbol** is in the global store, because the watchlist, header, summary and chart all need it. **Rule:** keep state as **close** to where it’s used as possible, and lift it to a global store only when distant components share it. `ChartPanel` is a **sibling** of `SymbolSummary` (which re-renders every tick), so price ticks never re-render the chart component.

9. Small but real details

- **Time zones:** Binance sends UTC milliseconds, and the library shows whatever time it’s given. We convert to seconds and **shift by the local offset**, so the axis shows your clock.
- **Price precision:** the axis uses 2 decimals for BTC and 4 for DOGE (`priceDecimals()`, shared with the watchlist formatting).
- **Fallback hosts:** REST tries `data-api.binance.vision` first (Binance’s market-data host, which allows browser CORS), then `api.binance.com`.
- **CORS:** a browser only lets JS read a response from another domain if that server sends `Access-Control-Allow-Origin`. Binance’s public market-data endpoints do.
- The small **TradingView logo** on the chart is the attribution the library’s license asks for.

Try it yourself

1. Look at the chart: 500 one-minute candles, and the last one **moves** every ~2s. When the minute ends, a new candle starts.
2. Click **5m / 1h / 1d**: a quick “Loading candles…”, then a different zoom level of history.
3. Click different coins quickly in the watchlist: the chart always ends on the coin you clicked last (abort working).
4. DevTools → **Network** → **Fetch/XHR**: see the `klines` request and its array-of-arrays JSON. Switch coins fast and some requests show as **(canceled)**.
5. Network → **WS**: a second socket carrying `kline` messages. Look at `"x": false` → `true` when a candle closes.
6. Drag the chart to scroll back, and use the wheel to zoom. Hover to see the crosshair with OHLC.

Resume mapping

> “candlestick charts with **TradingView Lightweight Charts**” → `CandleChart.tsx` “**REST** for history + WebSocket for live updates” → `binanceRest.ts` + the kline stream “**async/await**” → `fetchCandles` and `loadHistory`

Interview Qs

1. **How does your chart get its data?** It takes a REST snapshot of the last 500 candles, then a WebSocket kline stream. `update()` replaces the forming candle or appends a new one when the interval rolls over.
2. **Why not just use the WebSocket?** It only sends *new* changes. There’s no history, so the chart would start empty.
3. **What happens if the user switches symbols quickly?** The effect cleanup aborts the old fetch with `AbortController` and closes its socket, so stale data can’t render on the new chart.
4. **What if a live candle arrives before the history?** I buffer it, then replay it after `setData`, skipping anything older than the last history bar.
5. **Why `useRef` for the chart?** The library needs a real DOM node and manages its own canvas. Refs hold the DOM node and the chart instance across renders without causing re-renders.
6. **Why doesn’t the chart go through React state?** `series.update()` draws to the canvas directly. Re-rendering React components for each tick would be pure overhead.
7. **Does `fetch` throw on a 500?** No. It resolves, and you check `res.ok`. It only rejects on network errors or aborts.
8. **async/await vs `.then()` ?** They’re the same Promises underneath. async/await reads like synchronous code and uses `try/catch` for errors.
9. **What is CORS?** A browser rule: JS can read a cross-origin response only if the server allows that origin through headers. It’s enforced by the browser, not the server.